

Tutorials (CE/CZ1006)

Tut 9

Important concepts & methodology in this week

- *Computer arithmetic (Q9.1)*
 - *Floating & fixed point*
 - *Considerations in programming*
- *Pipelines (Q9.2)*
 - *Counting of cycles used*
 - *Data dependency*
 - *Delayed branching*

$$(-1)^S \times (1.F)_2 \times 2^{E - \text{Bias}}$$

Q9.1

Floating Point (IEEE754)

- Max Range $\approx 2 \times 2^{128}$ ($\sim 2^{128}$ to -2^{128}).
- Max Precision (near to zero) less than 2^{-126}

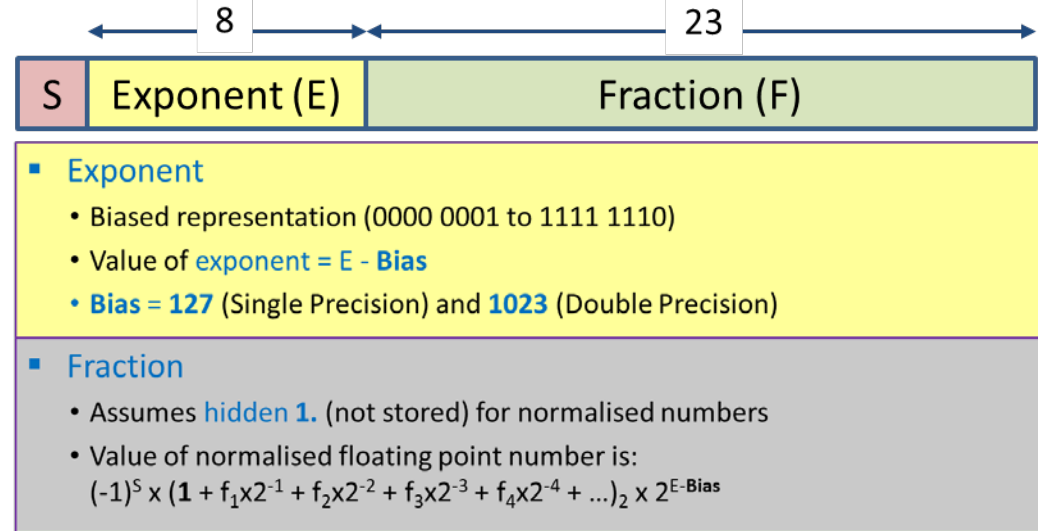
- Fixed Point – Max Range (Radix right of LSB, unsigned) $\approx 2^{32}$
- Max Precision (Radix left of MSB) 2^{-32}

- Floating point yield a larger range and better precision at small numbers with the same number of bits representation.

- One usually needs the best precision when the numbers are small.

- However, Fixed point number has the advantage of having uniform precision across entire range.

- Floating point number's precision changes across the range and the very coarse precision at the two end of the range may not be desirable to the intended algorithm.



Q9.2

Unsigned arithmetic used to maximise the range since the wire length cannot be negative

- 16-bit => maximum 0xFFFF
- Max length of one wire = 0x3FF => Max number of wire length that can be accumulated before the result will potentially overflow = $2^{16} / 2^{10} = 64$
- Math Expression = $\{(L[0] + \dots + L[63])/64 + (L[64] + \dots + L[127])/64 + (L[128] + \dots + L[191])/64 + (L[192] + \dots + L[255])/64\}/4$

Q9.3

I1 to I4: 7 cycles (I1 will take 4 cycles to pass through the pipeline)

I5 to I12. For the first four iterations, each iteration takes (8+2) cycles. This includes 2 cycles discarded in each iteration.

I5 to I12. Fifth iteration takes 8 cycles since the branch is not taken so no instructions are discarded.

I13 to I14: 2 cycles.

Total = $7 + (8+2)*4 + 8 + 2 = 57$ cycles.

```
MOV    R6, #0x900 ; I1
MOV    R5, #0      ; I2
MOV    R4, #0x800 ; I3
MOV    R3, #0x300 ; I4
Loop   LDR    R0, [R3] ; I5
        LDR    R1, [R4] ; I6
        ADD    R2, R1, R0 ; I7
        ADD    R5, R5, #1 ; I8
        ADD    R4, R4, #1 ; I9
        ADD    R3, R3, #1 ; I10
        CMP    R5, #5      ; I11
        BNE    Loop        ; I12
        ADD    R4, R4, R2 ; I13
        STR    R2, [R6]    ; I14
```

Figure 9.2

Q9.4

- I4 and I5. Data dependency. R3 register value is used in I5 before I4 completes loading R3 register. Resolved by
 - swapping I3 and I4, OR swapping I5 and I6, OR
 - inserting a NOP instruction between I4 and I5
- I6 and I7. Data dependency. R1 register value is used in I7 before I6 completes loading R1 register. Resolved by
 - swapping I7 and I8, OR
 - inserting a NOP instruction between I6 and I7
- I8 and I9. Data dependency. R5 register value is used in I9 before I8 completes loading R1 register. Resolved by
 - swapping I7 and I8, OR
 - inserting a NOP instruction between I8 and I9
- I5 and I12. Data dependency. I11 and I12 are delay slot instructions. During the first four iterations of the loop, I5 uses R3 before I12 can load the latest increment to R3. Resolved by
 - swapping I11 and I12, and add a NOP before I13.
- No branch conflicts since delayed branching is used.
- Assumption not stated in the question (should be stated) is that Status Bits are updated in the E stage, that why there are no data dependency conflicts between CMP and BNE instructions.

```
MOV    R6, #0x900 ; I1
MOV    R5, #0      ; I2
MOV    R4, #0x800 ; I3
MOV    R3, #0x300 ; I4
Loop   LDR    R0, [R3] ; I5
        LDR    R1, [R4] ; I6
        ADD    R2, R1, R0 ; I7
        ADD    R5, R5, #1 ; I8
        CMP    R5, #5      ; I9
        BNE    Loop      ; I10
        ADD    R4, R4, #1 ; I11
        ADD    R3, R3, #1 ; I12
        ADD    R4, R4, R2 ; I13
        STR    R2, [R6]   ; I14
```

Figure 9.3